🚀 [프론트엔드 가이드] 컴포넌트화 (Componentization)란 무엇인가?

> **문서 목표:** 단순히 디자인을 복사 붙여넣기 하는 것을 넘어, 역할과 책임을 명확히 나누는 '컴포넌트' 설계 방식을 이해하고 우리 프로젝트에 적용합니다.

---

1. 컴포넌트화의 핵심: 레고 블록 조립하기

우리가 컴포넌트를 만드는 이유는 **"똑같은 코드를 두 번 짜지 않기 위해서"**입니다. 버튼, 입력창(Input), 팝업(Modal) 등의 UI를 레고 블록처럼 미리 만들어두고, 필요한 화면(페이지)에서 블록을 꺼내 조립만 하는 것이 핵심입니다.

❌ 잘못된 접근 (파편화)

화면마다 `<button className="bg-green-500...">`, `<button className="bg-red-500...">` 처럼 매번 CSS를 새로 작성하는 방식입니다. 나중에 "우리 포인트 컬러 파란색으로 바꿀게요!" 하면 프로젝트 전체를 뒤져서 색상을 수정해야 하는 대참사가 발생합니다.

⭕ 올바른 접근 (단일 진실 공급원)

디자인(CSS)이 적용된 `<Button>` 파일을 딱 하나만 만들어두고, 모든 화면에서 이 파일을 불러와(Import) 사용합니다. 디자인 변경 시 이 파일 하나만 수정하면 전 화면에 일괄 적용됩니다.

---

2. 껍데기(UI)와 알맹이(Logic)의 분리 (Smart vs Dumb)

가장 중요한 규칙입니다. 컴포넌트는 크게 두 가지 역할로 나뉩니다.

- **Dumb 컴포넌트 (UI 부품 - 껍데기):**
  - 예: `Button.tsx`, `Input.tsx`
  - 자기 자신이 눌렸을 때 **'무슨 일이 일어나는지' 절대 몰라야 합니다.**
  - 백엔드 API 호출, 로그인 로직 등은 절대 이 안에 작성하지 않습니다. 오직 예쁘게 보여주는 역할만 합니다.
- **Smart 컴포넌트 (페이지/도메인 - 알맹이):**
  - 예: `ReservationPage.tsx`
  - 비즈니스 로직(API 통신, 상태 관리)을 모두 쥐고 있습니다.
  - Dumb 컴포넌트를 불러온 뒤, `onClick={예약함수}` 처럼 로직을 **주입(Props)** 해줍니다.

---

3. HTML 순정 기능 100% 활용하기 (Attributes 상속)

공통 컴포넌트를 만들 때, 기존 HTML 태그의 기능(`onClick`, `disabled` 등 수십 가지)을 일일이 새로 만들 필요가 없습니다. React가 제공하는 기본 타입을 상속받아 **'풀옵션 순정 차에 예쁜 도색만 새로 하는 방식'**을 사용합니다.

```
import type { ButtonHTMLAttributes } from "react";

// 기본 버튼의 모든 속성(onClick, disabled 등)을 그대로 물려받고(+ extends)
// 우리만의 디자인 속성(variant)만 추가합니다.
interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "danger" | "outline";
}

export const Button = ({ variant = "primary", ...props }: ButtonProps) => {
  return (
    <button className={`공통_CSS ${variant}에_따른_CSS`} {...props}>
      {/* 내용이 들어갈 자리 */}
    </button>
  );
};
```

---

4. 유연성의 핵심: 합성 (Composition과 children)

버튼 안에 글자만 들어갈지, 아이콘이 들어갈지, 로딩 스피너가 들어갈지 미리 예측해서 모든 경우의 수를 설정값(Props)으로 만들면 코드가 지저분해집니다. 대신, 컴포넌트의 속을 텅 비워두는 `children` 속성을 사용하여 사용하는 쪽에서 자유롭게 내용을 끼워 넣게 만듭니다.

[사용 예시 - 조립하는 쪽(Page)]

```
{/* 1. 글자만 넣기 */}
<Button variant="primary">예약하기</Button>

{/* 2. 아이콘과 함께 넣기 */}
<Button variant="outline">⭐ 즐겨찾기</Button>

{/* 3. 상태에 따라 로딩 스피너 넣기 */}
<Button variant="primary" disabled={isLoading}>
  {isLoading ? "⚡ 통신 중..." : "결제하기"}
</Button>
```

---

5. 실전 가이드: 어디까지 컴포넌트로 쪼개야 할까?

기능을 개발하다 보면 "이것도 컴포넌트로 빼야 하나?" 고민될 때가 있습니다. 아래 **3대 원칙**을 기준으로 판단하세요.

1. **재사용의 원칙 (The "Rule of 2")**
    - 똑같은 UI 덩어리가 우리 프로젝트 내에서 **2곳 이상에서 쓰일 때만** 공통 컴포넌트로 분리하세요. 한 페이지에서만 쓰는 거라면 굳이 분리하지 말고 페이지 안에 두어도 괜찮습니다.
2. **가독성의 원칙**

    - 파일 하나가 너무 길어져서(약 200줄 이상) 코드를 파악하기 힘들다면, 논리적인 덩어리(예: `<상단_충전소정보 />`, `<하단_리뷰목록 />`)로 잘라서 가독성을 높이세요.

3. **상태(Data) 최적화의 원칙**
    - 특정 데이터가 바뀔 때 화면 전체가 깜빡이지 않고 **딱 그 부분만 바뀌게 하고 싶을 때** (예: 충전기 사용 가능 뱃지) 컴포넌트로 작게 쪼개서 관리하세요.

6. 공통 부품 규격 사전 (Variant & Size)

컴포넌트를 호출할 때 사용하는 `variant`(테마)와 `size`(크기)는 팀원 간의 **'디자인 약속'**입니다. 상황에 맞게 알맞은 규격을 꺼내 쓰세요.

🎨 Variant (디자인 테마)

버튼이나 뱃지(Badge) 등이 어떤 성격의 액션인지 시각적으로 알려주는 역할입니다.

- `primary` **(메인 액션 / 포인트 컬러)**
    - **의미:** 이 화면에서 사용자가 **가장 먼저, 혹은 반드시 눌러야 하는 핵심 버튼**입니다.
    - **사용처:** [예약하기], [결제하기], [로그인], [작성 완료]
    - **디자인:** 눈에 가장 잘 띄는 꽉 찬 브랜드 컬러 (예: 초록색/파란색 바탕 + 흰 글씨)
- `secondary` **(보조 액션 / 무채색)**
    - **의미:** 메인 액션을 돕거나, 굳이 눈에 띌 필요가 없는 **일반적인 버튼**입니다.
    - **사용처:** [취소], [뒤로 가기], [닫기], [목록으로]
    - **디자인:** 튀지 않는 무채색 (예: 진회색/연회색 바탕 + 흰/검은 글씨)
- `danger` **(위험/경고 액션)**
    - **의미:** 누르면 데이터가 삭제되거나, 되돌릴 수 없는 **위험한 행동**을 할 때 경각심을 주는 버튼입니다.
    - **사용처:** [회원 탈퇴], [예약 취소], [기기 고장 신고], [게시글 삭제]
    - **디자인:** 가능한 빨간색 계열을 사용하여 위험함을 알림.
- `outline` **(강조된 보조 액션)**
    - **의미:** 배경은 투명하고 테두리만 있는 버튼으로, `secondary`보다는 중요하지만 `primary`보다는 덜 중요한 애매한(?) 위치에 사용합니다.
    - **사용처:** [★ 즐겨찾기], [필터 적용], [보조 옵션 선택]

📏 Size (크기)

사용되는 위치에 따라 버튼의 크기를 결정합니다. 패딩(Padding)과 폰트 크기가 세트로 움직입니다.

- `lg` **(Large):** 화면 하단에 꽉 차게 들어가는 제출/예약용 왕버튼
- `md` **(Medium):** 가장 기본이 되는 일반적인 폼(Form) 버튼 (기본값)
- `sm` **(Small):** 표(Table) 내부나 리스트 옆에 작게 붙는 수정/삭제 버튼